Blitz Planning -> Azure: Inventory & Migration Handover (for IT)

Prepared 2026-10-16. Technical handover for the Azure/IT administrator.

*The application currently runs entirely on **Netlify** (static hosting + serverless functions + Netlify Blobs). Goal: move it to the company's **Azure** environment.*

This document is self-contained: it lists everything the application needs, maps each piece to an Azure service, gives a concrete porting strategy, and flags the risks and decisions that are IT's to make.

---

1. Application at a glance

Property	Value
Type	Progressive Web App (PWA) + serverless HTTP API
Frontend	Static files in `public/` — **no build step**, served as-is; offline-capable via a service worker
Backend	**51 serverless functions** in `netlify/functions/` (Node.js, ES modules)
Data store	**Netlify Blobs**, one store named `blitz-data`, `consistency: 'strong'` (key-value JSON; **not** a SQL database)
Source control	GitHub, private repo `Brekke20/Blitz-Panning` (personal account)
Version	v2.1.6
Runtime	Node.js, ESM (`import` / `export`); no pinned engine — target **Node 20 LTS** on Azure
Deploy today	Netlify auto-builds/deploys on push to `main`
Tests	`node --test` (~871 unit tests) + Playwright e2e (~503 tests)

Architecture (current)

```
Browser (PWA, service worker, offline cache)
   |  HTTPS
   +- static assets ---------------> Netlify CDN  (public/)
   +- /api/*  --redirect--> /.netlify/functions/*  (51 functions)
                                  |
                                  +- Netlify Blobs  (store: blitz-data, strong)
                                  +- Zoho Desk API  (tickets + customer e-mail)
                                  +- TomTom API     (routing / geocoding / traffic)
                                  +- headless Chromium (PDF reports)
```

---

2. Full inventory (everything the planner needs)

2.1 Frontend (`public/`)

- One `index.html` plus ~180 JavaScript modules, CSS, `manifest.json`, `sw.js` (service worker).
- Served statically. The service worker caches a fixed shell list and specific CDN URLs; **cache version (`CACHE_NAME`) must be bumped on every release** or clients keep stale copies.
- Loads a few libraries from public CDNs at runtime (see 2.6).

2.2 Backend functions (`netlify/functions/`, 51 total)

Representative grouping (names are the `/api/<name>` endpoints):

Area	Functions
Auth / users	`auth-login`, `auth-ik`, `auth-uitloggen`, `auth-wachtwoord`, `auth-setup`, `auth-herstel`, `gebruikers`, `setup`
Planning / tickets	`tickets`, `plan`, `plan-datum`, `planning-sinds`, `planning-export`, `afspraken`, `availability`, `klantbeschikbaarheid`, `voorstel-status`, `confirm-afspraak`, `comment`

Area	Functions
Routing (TomTom)	`route`, `optimize`, `matrix`, `drukte`, `postcode`
Reports / Zoho	`propose`, `annuleer`, `send-rapport`, `rapport`, `rapport-archief`, `rapport-ontvangen`, `rapport-verzonden`, `rapport-verwerk-background`, `rapport-vangnet`, `mail-check`, `zoho-agenten`
Sales	`sales`, `sales-import`, `sales-opruimen`
Admin / misc	`dashboard`, `dashboard-instellingen`, `instellingen`, `prijzen`, `inventaris`, `fotos`, `activiteit`, `activiteit-opruimen`, `systeemstatus`, `client-log`, `testdata`, `spel`

Shared server code lives in `netlify/lib/` (~60 modules): auth, permissions (`rechten.js`), Zoho client (`zoho.js`), HTTP/CORS helpers (`http.js`), blob read-modify-write helper (`blob-wijzig.js`), the v1<->v2 adapter (`v2-adapter.js`), etc.

Handler shape (important for porting). Internally every handler is a plain function `(event, context) => { statusCode, headers, body }` (AWS-Lambda-style). A small adapter (`v2-adapter.js` -> `alsV2`) converts a web-standard `Request/Response` into that shape and back. **This means a single thin Azure adapter can wrap all existing handlers unchanged** — see §4.1.

2.3 Data store

- **Netlify Blobs**, store `blitz-data`, created with `consistency: 'strong'`.
- Key-value: each key holds a JSON document (planning per date, users, settings, availability, appointments, report register, activity log, game scores, …).
- Writes use an **optimistic-locking pattern implemented in app code** (`blob-wijzig.js`): because `@netlify/blobs` 8.2 has no conditional write, it writes then re-reads to detect conflicts and retries once (409 handling). Azure Blob Storage supports this **natively** via ETags (`If-Match`), which is cleaner and safer — see §4.2.
- **There is no external database to migrate — only this blob store.** But its contents are production data and must be copied over exactly.

2.4 External integrations (stay as-is, called from the backend)

Integration	Purpose	Auth
Zoho Desk	Read tickets; send proposal / cancellation / report e-mails to customers	OAuth2 with a long-lived **refresh token** (token cached ~55 min in-process)
TomTom	Travel times, route optimization, traffic, geocoding	API key

Outbound HTTPS to `*.zoho.com` / `*.zoho.eu` and `api.tomtom.com` must be allowed from the backend.

2.5 Secrets / configuration (environment variables)

Set as function app settings (ideally Key Vault references). None are in source control.

Variable	Purpose	Notes
`ZOHO_CLIENT_ID`	Zoho OAuth app id	sensitive
`ZOHO_CLIENT_SECRET`	Zoho OAuth app secret	sensitive
`ZOHO_REFRESH_TOKEN`	Zoho long-lived token	sensitive
`ZOHO_FROM_EMAIL`	From-address for customer mail	
`TOMTOM_API_KEY`	TomTom key	sensitive
`SESSIE_GEHEIM`	HMAC secret that signs the session cookie	sensitive — rotating it logs everyone out

Variable	Purpose	Notes
`CONFIRM_LINK_SECRET`	Signs appointment-confirmation links	sensitive
`PLANNING_EXPORT_API_KEY`	Auth for the external planning-export endpoint	sensitive
`URL`	The app's own public base URL	**Netlify sets this automatically; on Azure it must be set explicitly** (used in customer links)
`BLITZ_VANGNET_ZELF`	Toggle for the safety-net self-check	

2.6 Runtime CDN dependencies (loaded by the browser, not bundled)

- **Leaflet 1.9.4** (map) — `cdnjs.cloudflare.com`
- **signature_pad 4.1.7** (signature capture) — `cdnjs.cloudflare.com`
- **ExcelJS 4.4.0** (Excel export, lazy-loaded) — `cdn.jsdelivr.net`
- **Google Fonts** — `fonts.googleapis.com` , `fonts.gstatic.com`

If the corporate network blocks these hosts, the map / export / fonts break. Option: self-host these files instead.

2.7 Special backend behaviours (the migration hotspots)

Behaviour	Function	Detail
PDF generation	`rapport`	Launches **headless Chromium** via `puppeteer-core` + `@sparticuz/chromium-min` (built for AWS Lambda). The hardest part to port — see §3.1. Report PDF template assets are bundled via `included_files` .
Scheduled job	`rapport-vangnet`	Cron `*/5 * * * *` (every 5 min), 30 s timeout — safety-net for report mails.
Background job	`rapport-verwerk-background`	Netlify "background function" (returns 202, keeps working; triggered with an internal token).
Long timeouts	`rapport` , `send-rapport` , `propose` , `annuleer` , `planning-sinds` , `planning-export` , `mail-check` , `rapport-ontvangen` , `sales` , `postcode` , `sales-import` , `dashboard`	26 s; `rapport-vangnet` 30 s; `confirm-afspraak` 15 s. Default elsewhere.
API routing	—	`/api/*` -> `/.netlify/functions/:splat` (Netlify redirect).

---

3. Target Azure architecture

Recommended, in order of "closest fit + least friction":

Today (Netlify)	Recommended Azure service
Static site (`public/`)	**Azure Static Web Apps** (Standard tier)
51 serverless functions	**Azure Functions** — a **standalone Function App** *linked* to the Static Web App ("Bring your own functions"), Node 20, v4 programming model
`/api/*` routing	SWA routing + `staticwebapp.config.json` ; the linked Function App answers `/api/*`
Netlify Blobs (`blitz-data`)	**Azure Blob Storage** (one container; JSON blobs; ETag `If-Match` for concurrency)
Scheduled job (cron)	**Azure Functions Timer trigger** (`0 */5 * * * *`)
Background job	**Azure Functions Queue trigger** (Azure Storage Queue) — enqueue + process
Chromium/PDF	**Azure Container Apps** (or a container-based Premium Function) running **Playwright** — see §3.1

Today (Netlify)	Recommended Azure service
Secrets (env vars)	**Azure Key Vault** + app settings as Key Vault references; backend reads Blob/Vault via **Managed Identity** (no connection strings in code)
Auto-deploy on push	**GitHub Actions** (SWA ships a workflow; Function App via `azure/functions-action`)
Logs/metrics	**Application Insights**
TLS + domain	SWA/Front Door **managed certificate** on the custom domain

> **Why standalone Functions, not SWA "managed" functions:** SWA's *managed* functions are **HTTP-only** and capped in duration — they do **not** support Timer or Queue triggers, which this app needs (cron + background). A **standalone Function App linked to SWA** removes those limits and gives full control over timeouts, scaling and triggers. This is the main architectural decision for IT.

Timeouts on Azure

- Consumption plan: `functionTimeout` default 5 min, configurable to **10 min** — comfortably covers the 26–30 s needs.
- HTTP requests through the gateway have a ~230 s idle limit — also fine.
- Only the PDF path needs more than Consumption can comfortably give (large browser binary, cold starts) -> Premium/Container.

---

4. Concrete porting strategy

4.1 The functions: one adapter wraps them all

The app already separates business logic from transport (`event -> {statusCode, headers, body}`), with `alsV2` adapting web `Request/Response` to it. Mirror that for Azure: write an `alsAzure(handler)` that turns an Azure `HttpRequest` into the same `event` object and the `{statusCode, headers, body}` result into an `HttpResponse` . Then each function's export changes from the Netlify wrapper to the Azure one; **the handler bodies and all of `netlify/lib/` stay unchanged.** This keeps the diff small and testable and preserves the existing unit tests.

Also needed:

- ESM packaging: add `"type": "module"` (or `.mjs`) so Azure Functions loads the modules the same way Netlify's bundler does.
- Replace the Netlify `included_files` mechanism for the PDF template assets with files shipped in the deployment package.
- Replace `process.env.URL` with an explicit app setting.

4.2 The data layer: one module to swap

All blob access goes through a small set of helpers (`getStore({name:'blitz-data', consistency:'strong'})` and `blob-wijzig.js`). Implement an Azure Blob Storage equivalent (`@azure/storage-blob`) behind the same interface:

- `get(key)` -> download blob (JSON), return value + ETag.
- `set(key, value)` -> upload with `If-Match: <etag>` (or `If-None-Match: *` for create) -> **native optimistic concurrency**, replacing the read-after-write retry hack.
- Strong read-after-write is guaranteed per blob in Azure Blob Storage.

Swap this one module and the 51 functions follow automatically.

4.3 Data migration

1. Export every key from Netlify Blobs store `blitz-data` (Netlify Blobs API/CLI: list + get).
2. Upload each as a blob into the Azure container (preserve keys/paths exactly).
3. Diff/verify counts and a sample of documents.
4. Do a **final re-sync during the cutover window** so no writes are lost between export and go-live.

4.4 PDF generation (§3.1)

`@sparticuz/chromium` targets AWS Lambda; it is not a drop-in on Azure. Recommended: a small **Playwright** service in **Azure Container Apps** (official Playwright Docker image already contains the browser), exposed internally and called by `send-rapport` / `rapport`. Alternative: a **container-based Premium Function**. Validate this early and in isolation — it is the single biggest unknown.

---

5. Risks & watch-outs

1. **PDF/Chromium (highest effort & risk).** Must be rebuilt (Playwright in a container). Budget real time for it.
2. **Data migration, not just code.** All production state is in Netlify Blobs; it must arrive complete and correct, with a final cutover re-sync.
3. **No double-running during cutover.** While either side can send Zoho customer mail, old (Netlify) and new (Azure) must **not** be live simultaneously -> one clear switch-over, with a rollback plan (keep Netlify ready to re-enable).
4. **SWA managed functions won't do timer/queue** -> use a standalone Function App (decision in §3).
5. **Strong consistency / concurrency.** Re-implement the optimistic lock with ETags carefully; a mistake risks lost updates when two users save at once.
6. ** `URL` setting.** Must be set explicitly on Azure; it feeds customer-facing links (proposals, confirmation links) — wrong value breaks those links.
7. **Secrets handling.** Zoho/TomTom/session secrets are sensitive; move them straight into Key Vault, never via mail/chat. Decide whether to rotate (rotating `SESSIE_GEHEIM` logs everyone out; rotating `ZOHO_REFRESH_TOKEN` requires re-consent).
8. **Repo ownership.** Source is on a **personal** GitHub account (`Brekke20`). Decide deploy access / move under a company org / use a deploy token for Azure.
9. **Service worker cache.** Bump `CACHE_NAME` at cutover or users stay on the old build; also plan for the origin/domain change.
10. **Outbound network.** Backend needs egress to Zoho, TomTom and (for the browser) the CDN/font hosts. If the corporate network restricts egress, allow-list them or self-host the CDN libraries.
11. **Cost.** Netlify is largely free today; Azure bills per use (Functions executions, Blob storage/ops, Container Apps for PDF, possibly Premium plan, App Insights, Front Door). Get an estimate up front.
12. **Region / data residency.** Pick an EU region (e.g. West Europe) for customer data; confirm it matches the Zoho data-centre region.

---

6. Recommendations for a smooth migration

1. **Spike first, don't move everything at once.** Stand up a test SWA + a standalone Function App + a test Blob container with a handful of simple functions. Learn Azure's behaviour before touching real data.
2. **Do the data layer as one central change** (§4.2) and test it in isolation first; the rest follows.
3. **Tackle PDF generation as a separate sub-project**, proven early and independently.
4. **Keep the existing test suites green** at each step — they are the proof that Azure behaves identically to Netlify.
5. **Use Managed Identity + Key Vault** from the start (no secrets in code or workflow logs); wire up Application Insights for visibility.
6. **Write a cutover runbook**: timing, who disables Netlify, how to verify Azure (login, a test ticket, a test proposal mail, a test PDF), and the rollback path.
7. **Infrastructure as code** (Bicep/Terraform) so the environment is reproducible (staging + production).
8. **Nothing goes live without Brent's approval**, especially the moment customer mail starts flowing through Azure.

---

7. Open decisions for IT / the business

- Which Azure **subscription / resource group**, and who owns/pays for it?
- **Plan tier**: SWA Standard + Functions Consumption (likely enough) vs Premium (for the PDF path / always-warm)?
- **Repo access**: keep on `Brekke20` , move to a company org, or deploy via token?
- **Region** (EU / West Europe), and does it match Zoho's data centre?

- **Secret policy**: Key Vault, rotation, who holds the Zoho/TomTom credentials?
- **Networking**: open egress, or restricted + allow-list / self-hosted CDN assets?
- **Budget** ceiling and expected monthly cost.

Bottom line

Feasible and a good fit: **Azure Static Web Apps + a standalone Azure Functions app + Azure Blob Storage**, with a small **Container App (Playwright)** for PDFs. The code ports with a thin transport adapter and one data-layer swap; the real work and risk are the **PDF service** and the **complete, lossless data migration**.